

NumPy Beginner + Intermediate Complete Notes (Baap Ji Edition)

This PDF contains definitions, explanations, use cases, and examples for the major NumPy topics covered.

1. Array Creation

`np.array()`: Creates a NumPy array from Python lists. Arrays are faster and more memory efficient than Python lists.

Example: `np.array([1,2,3])`

`np.zeros()`, `np.ones()`, `np.full()`, `np.empty()`, `np.arange()`, `np.linspace()` are used to quickly generate arrays for data science and machine learning workflows.

2. Array Attributes

`ndim` = number of dimensions. `shape` = size of each dimension. `size` = total elements. `dtype` = data type. `itemsize` = bytes used by one element.

3. Indexing and Slicing

Indexing accesses a specific element. Slicing extracts a range of elements. Negative indexing starts from the end. Slicing syntax: `start:stop:step`.

4. Reshaping

`reshape()` changes array shape without changing data. `flatten()` creates a copy in 1D. `ravel()` creates a flattened view when possible. `transpose()` and `.T` swap rows and columns.

5. Arithmetic Operations

`+`, `-`, `*`, `/`, `//`, `%`, `**` operate element-wise. NumPy performs vectorized computation, avoiding loops and improving performance.

6. Broadcasting (Important)

Broadcasting allows arrays of different shapes to work together. Compare dimensions from right to left. Dimensions are compatible if they are equal or one of them is 1. Missing dimensions are treated as 1.

Example: (3,4) and (4,) becomes (3,4) and (1,4). Broadcasting succeeds.

Common confusion: Arrays do NOT both need a 1 somewhere. The rule is checked pair-by-pair for every dimension.

7. Comparison Operators

`==`, `!=`, `>`, `<`, `>=`, `<=` return Boolean arrays. Every element is compared independently.

8. Boolean Masking

A Boolean mask is an array of True/False values used to filter data. Example: `a[a > 10]`. Use `&`, `|`, and `~` instead of `and`, `or`, `not`.

9. Iteration

for loops iterate through arrays. `np.nditer()` is useful for arrays with many dimensions. `enumerate()` provides index and value together.

10. Random Module

`rand()` creates random floats from 0 to 1. `randn()` samples from a normal distribution. `randint()` creates random integers. `choice()` selects random values. `seed()` ensures reproducible results.

11. Sorting and Searching

`sort()` returns sorted values. `argsort()` returns indices that would sort the array. This was the most confusing topic.

Easy explanation of `argsort()`: First sort the values mentally. Then ask: where did each sorted value come from in the original array? Return those indices.

Example: `[30,10,20]` -> sorted values `[10,20,30]`. Original indices are `[1,2,0]`. Therefore `argsort()` returns `[1,2,0]`.

`argmax()` returns index of largest value. `argmin()` returns index of smallest value. `where()` returns indices matching a condition.

12. Aggregation Functions

`sum()` total of values. `mean()` average. `median()` middle value after sorting. `min()` smallest. `max()` largest. `std()` measures spread. `var()` is variance and equals standard deviation squared.

13. Array Joining

`concatenate()` joins arrays along existing dimensions. `hstack()` joins horizontally. `vstack()` joins vertically. `stack()` creates a new dimension.

Interview Tip: `concatenate` merges existing dimensions, while `stack` creates a new dimension.

14. Array Splitting

`split()` divides arrays equally. `hsplit()` splits columns. `vsplit()` splits rows.

15. Linear Algebra

`dot()` computes dot products. `matmul()` and `@` perform matrix multiplication. `*` is element-wise multiplication and is NOT matrix multiplication.

Determinant (`det`) indicates whether a matrix can be inverted. If determinant is 0, inverse does not exist. `inv()` computes matrix inverse.

Final Advice

You have completed the Beginner and Intermediate NumPy syllabus. The next stage is Advanced NumPy (views vs copies, advanced indexing, vectorization internals, performance optimization) or moving into Pandas and Machine Learning.